

Submodular Order Functions and Assortment Optimization

Rajan Udawani^a

^aIndustrial Engineering and Operations Research, University of California, Berkeley, California 94720

Contact: rudwani@berkeley.edu,  <https://orcid.org/0000-0002-2112-4876> (RU)

Received: December 13, 2021

Revised: November 25, 2023

Accepted: January 29, 2024

Published Online in Articles in Advance:
April 1, 2024

<https://doi.org/10.1287/mnsc.2021.04108>

Copyright: © 2024 INFORMS

Abstract. We define a new class of set functions that, in addition to being monotone and subadditive, also admit a very limited form of submodularity defined over a permutation of the ground set. We refer to this permutation as a submodular order. This class of functions includes monotone submodular functions as a subfamily. We give fast algorithms with strong approximation guarantees for maximizing submodular order functions under a variety of constraints and show a nearly tight upper bound on the highest approximation guarantee achievable by algorithms with polynomial query complexity. Applying this new notion to the problem of constrained assortment optimization in fundamental choice models, we obtain new algorithms that are both faster and have stronger approximation guarantees (in some cases, first algorithm with constant factor guarantee). We also show an intriguing connection to the maximization of monotone submodular functions in the streaming model, where we recover best known approximation guarantees as a corollary of our results.

History: Accepted by Chung Piaw Teo, optimization.

Funding: This work was supported by the NSF Division of Civil, Mechanical, and Manufacturing Innovation [Grant 2340306] and Google Research Scholar Program.

Supplemental Material: The online appendices are available at <https://doi.org/10.1287/mnsc.2021.04108>.

Keywords: submodular order • assortment optimization • approximation algorithms

1. Introduction

Given a ground set N , consider the problem of finding a (feasible) subset S that maximizes a set function $f: 2^N \rightarrow \mathbb{R}$ that satisfies the following properties,

Monotonicity: $f(A) \geq f(B) \quad \forall B \subseteq A$,

Submodularity: $f(A \cup \{e\}) - f(A) \leq f(B \cup \{e\}) - f(B)$
 $\forall B \subseteq A \subseteq N, e \in N \setminus A$.

Since the work of Nemhauser et al. (1978), this problem has received significant and continued interest in operations research, computer science, and many other communities due to a variety of modern applications, such as feature selection (Das and Kempe 2011, Wei et al. 2015), sensor placement (Leskovec et al. 2007, Krause et al. 2008a), influence maximization (Krause et al. 2008b), data summarization (Badanidiyuru et al. 2014), and many others. Despite this richness of submodular functions, there are important subset selection problems where the objective (f) is *not* submodular.

Of particular interest to us is the setting of *assortment optimization* where the objective is not submodular. The problem arises widely in many industries including retailing and online advertising, where given a universe of substitutable products, the seller wishes to

select a subset of products (called an assortment) with maximum expected revenue. The effect of substitution between products is captured by a *choice model* $\phi: N \times 2^N \rightarrow [0, 1]$. Given assortment S , the probability that customer chooses product $i \in S$ is given by $\phi(i, S)$. Customer may choose an *outside option* that is not in S with probability $1 - \sum_{i \in S} \phi(i, S)$. Given (fixed) prices $(r_i)_{i \in N}$, the expected revenue is

$$R_\phi(S) = \sum_{i \in S} r_i \phi(i, S).$$

Given the choice model ϕ , we are interested in the problem of finding the revenue optimizing assortment subject to certain types of constraints that we will discuss shortly. For a general choice model, Aouad et al. (2018) show that even the unconstrained assortment optimization problem is hard to approximate. The optimization problem is more tractable for commonly used simple choice models such as the multinomial logit (MNL) choice model (defined in Section 2.1), but the revenue objective is not submodular even in this special case. The objective is also nonmonotone, but one can transform it into a monotone objective (Section 2.2). Given this intractability and lack of general structure, the field of assortment optimization has evolved through algorithms that are tailored for individual families of choice models.

Motivated by this, we are interested in finding a unifying structural property (for set functions) that leads to computationally tractable optimization problems that capture both monotone submodular function maximization and assortment optimization problems as special cases. As a starting point, consider monotone functions that satisfy the following mild condition:

$$\text{Subadditivity : } f(A) + f(B) \geq f(A \cup B) \quad \forall A, B \subseteq N.$$

Every nonnegative submodular function is subadditive and the revenue function in assortment optimization is also subadditive for a general family of random utility-based choice models (Kök et al. 2008, Berbeglia and Joret 2020). Using the (standard) normalization $f(\emptyset) = 0$, monotonicity implies that f is nonnegative. Given a monotone subadditive function f , we are interested in solving

$$\arg \max_{S \in \mathcal{F}} f(S),$$

where $\mathcal{F}$ corresponds to one of the following types of constraints that commonly arise in applications:

Cardinality (k)	$\{S \mid S \leq k\},$
Budget/Knapsack ($\{b_i\}_{i \in N}, B$)	$\{S \mid b(S) \leq B\},$
Matroid ($\mathcal{I}$)	$\{S \mid S \in \mathcal{I}\},$

where $b(S) = \sum_{i \in S} b_i$ and $\mathcal{I}$ is the family of independent sets of a matroid. Both budget and matroid constraints include cardinality constraint as a special case. In the absence of a constraint, the ground set N maximizes function value (due to monotonicity). However, constrained optimization is computationally intractable for monotone subadditive functions even with a cardinality constraint.

Theorem 1 (Adapted from Theorem 6.1 in Dobzinski et al. 2010). *Any algorithm that makes polynomial number of queries (in size n of ground set), cannot have approximation guarantee better than $n^{-\Omega(1)}$ for maximizing a monotone subadditive function subject to cardinality constraint.*

In light of this impossibility result, we introduce a new structural property, namely, a *submodular order* and show that it drastically alters the optimization landscape by allowing efficient algorithms with constant factor guarantee for all constraint families of interest. To define this new notion we introduce some notation. Given a permutation π over N , we index elements in the order given by π . Let $r_\pi(S)$ denote the element in S with the largest index and $l_\pi(S)$ the element with the smallest index. It helps to visualize the indexing as an increasing order from left to right; $r_\pi(S)$ is the rightmost element of S and $l_\pi(S)$ is the leftmost. We define the marginal value function

$$f(X|S) = f(X \cup S) - f(S).$$

Submodular order: A permutation π of elements in N is a submodular order if for all sets $B \subseteq A$ and C to the right of A , that is, $l_\pi(C) > r_\pi(A)$, we have

$$f(C|A) \leq f(C|B). \quad (1)$$

We refer to a function f as a *submodular order function* if there exists a submodular order for the function. We refer to function with submodular order π simply as π -submodular ordered. Our main focus is on functions for which a submodular order is known but our algorithmic results extend to certain settings where a submodular order may not be known or may not even exist (discussed further in Section 3.3).

Comparison with submodular functions: For a submodular function f , Inequality (1) holds for *every* set C . In fact, it can be shown that a function is submodular if and only if every permutation of the ground set is a submodular order. From an optimization standpoint, there is a substantial difference between submodularity and submodular order. Consider the classic work of Nemhauser et al. (1978), which showed that the greedy algorithm that iteratively builds a solution by adding an element with the largest marginal value, achieves a guarantee of $(1 - 1/e)$ for maximizing monotone submodular functions subject to cardinality constraint. The following example demonstrates that greedy can be arbitrarily bad even for a simple submodular order function.

Example 1. Consider a ground set $\{1, \dots, 2k+1\}$. Let singleton values $f(\{e\})$ equal one for all *good* elements $e \in \{1, \dots, k\}$, equal ϵ for all *poor* elements $e \in \{k+1, \dots, 2k\}$ and finally, let $f(\{2k+1\}) = 1 + \epsilon$. Let the first $2k$ elements be modular, that is, the value of a set $S = S_1 \cup S_2$ with subset S_1 of good elements and S_2 of poor elements is simply $|S_1| + \epsilon|S_2|$. Finally, let $f(S_1 \cup S_2 \cup \{2k+1\}) = \max\{|S_1|, 1 + \epsilon\} + \epsilon|S_2|$ for every subset S_1 of good elements and S_2 of poor elements. It can be verified that this function is monotone subadditive and the natural indexing $\{1, \dots, 2k+1\}$ is a submodular order. The set of all good elements has value k , and this is the optimal set of cardinality k for every $k \geq 2$. However, the greedy algorithm would first pick element $2k+1$ and subsequently pick $k-1$ poor elements resulting in a total value of $1 + \epsilon k$. For $\epsilon \rightarrow 0$, this is only a trivial $1/k$ approximation of the optimal value.

What about other algorithms for submodular maximization? One may wonder if a greedy algorithm that chooses m elements at each step for $m \geq 2$, admits better performance. The previous example can be easily modified to show that this family of algorithms is arbitrarily bad in the worst case for any constant m (Online Appendix A). The more general schema of *continuous* greedy algorithms (Calinescu et al. 2011, Feldman et al. 2011) is similarly ineffective.

Local search is another well studied family of algorithms for submodular maximization (Feige et al. 2011). These algorithms incrementally improve the value of a solution by swapping elements. For submodular order functions, local search fails to improve the value of a poor solution. In the previous example, performing local search by swapping one element at a time (with the objective of improving function value) will not find any improvement on the set $\{k+1, \dots, 2k+1\}$. This set is a local maxima with value $1/k$ of the optimal. More generally, natural modifications of the example show that local search over m -tuples is arbitrarily bad in the worst case for any constant m (Online Appendix A). We now discuss our main algorithmic results for submodular order maximization and its applications.

1.1. Our Contributions

We start by describing our main results for submodular order maximization. The algorithms referenced in this section are presented subsequently in Section 3. All our approximation results hold more strongly under a milder version of submodular order, called *weak submodular order*, defined as follows.

Weak submodular order and π nested sets: Sets $B \subseteq A$ are π -nested if the left most element of $A \setminus B$ is to the right of B , that is, $l_\pi(A \setminus B) > r_\pi(B)$. An order π is a weak submodular order if $f(C|A) \leq f(C|B)$ for all π -nested sets $B \subseteq A$ and set C with $l_\pi(C) > r_\pi(A)$.

The weak submodular order property does not impose any requirements on sets A, B that are not π -nested. Because this is a milder condition, the resulting family of functions is broader than (strong) submodular order functions. An algorithm with guarantee α for weak submodular order functions is also an α approximation for submodular order functions. Conversely, the upper bound on approximation guarantee for submodular order functions also applies to functions with weak submodular order. It is not meant to be obvious a priori but we establish that, in general, the stronger notion does not offer any benefits in terms of approximation guarantee. We establish the following approximation guarantees. Table 1 provides a summary.

Table 1. Results for Constrained Maximization of (Weak) Submodular Order Functions

	Lower bound	Upper bound	No. of queries
Cardinality	$0.5 - \epsilon$		$O(\frac{n}{\epsilon} \log k)$
Budget/knapsack	$0.5 - \epsilon$	0.5	$O(\frac{1}{\epsilon} n^{1+\frac{1}{\epsilon}} \log n)$
Matroid	0.25		$O(nd)$

Notes. n denotes the size of the ground set, k the cardinality parameter, and d is the matroid rank. For budget constraint, we also show a faster $1/3 - \epsilon$ approximation. Approximation guarantees hold for weak (and strong) submodular order functions and the upper bounds hold for strong (and weak) submodular order functions.

Theorem 2. For cardinality constrained maximization of a monotone subadditive function f with a (known) weak submodular order, Algorithm 1 is $(1 - \epsilon)0.5$ approximate with $O(\frac{n}{\epsilon} \log k)$ oracle queries, for any choice of $\epsilon \in (0, 1)$.

Theorem 3 (Upper Bound). Given a monotone and subadditive function with (strong) submodular order on a ground set of size n , any algorithm for the cardinality constrained maximization problem that makes at most $\text{poly}(n)$ queries cannot have approximation guarantee better than $0.5 + \epsilon$ for any $\epsilon > 0$.

More generally, we have the following approximation results for budget and matroid constraint. In the corollary of Theorem 3, no efficient algorithm can have approximation guarantee strictly better than 0.5 for maximization subject to these constraints.

Theorem 4. For budget constrained maximization of a monotone subadditive function f with a (known) weak submodular order,

- (i) Algorithm 3 is $(1 - \epsilon)/3$ approximate with query complexity $O(\frac{n}{\epsilon} \log n)$, $\forall \epsilon \in (0, 1)$.
- (ii) Algorithm 5 is $0.5 - \epsilon$ approximate with query complexity $O(\frac{1}{\epsilon} n^{1+\frac{1}{\epsilon}} \log n)$, $\forall \epsilon \in (0, 0.5)$.

Theorem 5. Algorithm 7 is 0.25 approximate with query complexity $O(nd)$ for the problem of maximizing a monotone subadditive function f with a known (weak) submodular order, subject to a matroid constraint with matroid rank d .

Remarks. We prove Theorems 2, 4, and 5 in Section 4. The proof of Theorem 3 is given in Online Appendix B. Given a noisy value oracle $\hat{f}$ for a submodular order function f , such that $(1 - \delta)f(S) \leq \hat{f}(S) \leq (1 + \delta)f(S) \forall S \subseteq N$, we show that our approximation guarantees are reduced by an additional multiplicative factor of $(1 - O(\frac{n\delta}{1-\delta}))$. This property is important in applications such as assortment optimization, where the value oracle is implemented by solving an optimization problem (Section 2.2). We formalize this observation with the analysis of our algorithms.

1.2. Outline for the Rest of the Paper

In Section 2, we present applications of submodular order function maximization. First, we discuss new results for assortment optimization. Then, in Section 2.3, we discuss connections with streaming maximization of submodular functions. We present our algorithms and the underlying insights in Section 3. This is followed by analysis and proofs of our main results for submodular order maximization in Section 4, where we start with useful bounds for (weak) submodular order functions that drive the analysis of our algorithms. In Section 4.1, we prove Theorem 2 for the cardinality constrained problem. Section 4.2 presents the analysis of Algorithms 3 and 5 (Theorem 4) for budget

constraint. In Section 4.3, we establish Theorem 5 for matroid constraint. We give the proof of the upper bound (Theorem 3) for (strong) submodular order functions in Online Appendix B. In Section 5, we discuss the results from a small numerical experiment for Algorithm 1. Finally, Section 6 concludes the discussion with several directions for further research. Proofs for main results on assortment optimization are included in Online Appendix C.

2. Applications of Submodular Order Functions

In this section, we discuss application of submodular order function maximization to assortment optimization (Section 2.2) and to streaming maximization of submodular functions (Section 2.3).

2.1. Constrained Assortment Optimization

We start by providing background on assortment optimization and discussing related work. Recall that in assortment optimization, we assume that a choice model $\phi : N \times 2^N \rightarrow [0, 1]$ is given, and we are interested in the problem of finding the revenue optimizing assortment subject to some constraint. Given assortment S , the probability that customer chooses product $i \in S$ is given by $\phi(i, S)$. Customer may choose an *outside option* (not in S) with probability $1 - \sum_{i \in S} \phi(i, S)$. Given (fixed) prices $(r_i)_{i \in N}$, the expected revenue is

$$R_\phi(S) = \sum_{i \in S} r_i \phi(i, S).$$

Of the many choice models that have been introduced in the literature, we are interested in the following well-studied choice models.

Multinomial logit choice model (MNL) (Bradley and Terry 1952, Luce 1959, McFadden 1973, Plackett 1975): This model is defined by parameters $v_i \geq 0$ for $i \in N$. $v_0 \geq 0$ denotes the parameter for the outside option. The probability that a customer chooses product i from assortment S is proportional to v_i . Formally,

$$\phi(i, S) := \frac{v_i}{v_0 + \sum_{e \in S} v_e}.$$

Previous results for assortment optimization: The assortment optimization problem under MNL choice is very well studied. The objective in this problem is nonmonotone (and not submodular). Talluri and Van Ryzin (2004) showed that the unconstrained problem can be solved optimally and the optimal solution includes all products above a price threshold. Rusmevichientong et al. (2010) gave a polynomial time algorithm for the cardinality constrained problem. Sumida et al. (2021) and Avadhanula et al. (2016) showed that the MNL optimization problem can be solved optimally in polynomial time under totally unimodular constraints (TUM).

Désir et al. (2021) showed that the budget constrained version is NP hard, and they gave a Fully Polynomial Time Approximation Scheme (FPTAS) for the problem.

Mixture of MNL with Customization (El Housni and Topaloglu 2023): Consider a population given by m types of customers. Each customer chooses according to a MNL choice model that depends on their type. The choice model of the population is described by a mixture of MNLs (MMNL) choice models (McFadden and Train 2000). Customer type is revealed on arrival, and we offer a *customized* assortment based on the type. An offered assortment can be any subset of the products that we keep in our selection and suppose that we can keep at most k different products in the selection. Suppose that a random customer is type $j \in [m]$ with probability α_j . Thus, we would like to select at most k products to maximize

$$\max_{S, |S| \leq k} \sum_{j \in [m]} \alpha_j \max_{X \subseteq S} R_{\phi_j}(X),$$

where ϕ_j is the MNL choice model for customer type j . A $\frac{1}{m}$ approximation for this problem can be obtained simply by solving a cardinality constrained MNL assortment problem for each type separately and picking the best of these solutions. El Housni and Topaloglu (2023) gave a substantially stronger $\frac{\Omega(1)}{\log m}$ approximation for this problem and showed that it is NP hard to approximate the optimal assortment better than $(1 - \frac{1}{e} + \epsilon)$. For constant m , they gave a FPTAS. The unconstrained version of this problem (where S can be any subset of N) can be solved simply by taking the union of optimal unconstrained assortments for each type of MNL.

Markov choice model (Blanchet et al. 2016): In this model, the customer choice process is described by a discrete Markov chain on the state space of products (and the outside option). Given an assortment S , the customer starts at product $i \in N$ with probability λ_i . A customer at a product $i \notin S$, transitions to product $j \in N$ with probability ρ_{ij} (independent of their actions prior to i). The random process terminates when the customer reaches a product in $S \cup \{\emptyset\}$, which is their final choice. This model generalizes a wide array of choice models (Blanchet et al. 2016).

Previous results for assortment optimization: Blanchet et al. (2016) introduced this model and gave a polynomial time algorithm for unconstrained optimization. Désir et al. (2020) consider the problem under cardinality and budget constraints. They show that the problem is APX hard under cardinality constraint and inapproximable under TUM constraints (sharp contrast with MNL). They obtain a $0.5 - \epsilon$ approximation for the cardinality constrained problem and a $1/3 - \epsilon$ approximation for the budget constrained version.

MNL, MMNL, and Markov choice models have received significant attention in the assortment optimization

literature and even the simpler of these models, that is, MNL, is very useful for modeling choice behavior in practice (Feldman et al. 2022a).

2.2. New Results for Assortment Optimization

An obvious obstacle in the application of our results is the absence of monotonicity in the revenue objective. We solve this issue by redefining the objective as follows:

$$f_\phi(S) = \max_{X \subseteq S} R_\phi(X).$$

We show that the function f_ϕ is monotone and subadditive under the following (mild) condition:

$$\text{Substitutability: } \phi(i, S \cup \{j\}) \leq \phi(i, S), \quad \forall i \in S, j \notin S.$$

MNL, MMNL, and Markov models all satisfy this condition. To evaluate $f_\phi(S)$, we need to solve an unconstrained assortment optimization problem on the reduced ground set S . Recall that previous work gives polynomial time algorithms for unconstrained optimization under the choice models of interest to us. Therefore, we have efficient implementation of the value oracle for f_ϕ . More generally, if a choice model ϕ is such that f_ϕ has a submodular order but the unconstrained assortment problem only admits a $1 - \delta$ approximation (instead of an efficient optimal algorithm), then the approximation guarantees are reduced by a multiplicative factor of $(1 - O(\frac{\delta}{1-\delta}))$. This factor is negligible for small δ , and this can be useful in case the unconstrained assortment problem admits a FPTAS.

2.2.1. Results for MNL and MMNL with Customization. When ϕ is MNL, we show that sorting products in descending order of price (breaking ties arbitrarily) is a (strong) submodular order for f_ϕ . Consequently, all our results for constrained optimization of submodular order apply directly. More importantly, using the fact that submodular order functions are closed under addition (see Remark 1), we immediately obtain the first constant factor approximation for the problem of assortment optimization with customization under MMNL choice.

Remark 1. Given p monotone subadditive functions $f_1, \dots, f_m$ with the same submodular order π and non-negative real values $\alpha_1, \dots, \alpha_m$, the function $\sum_{j \in [m]} \alpha_j f_j$ is also monotone subadditive with submodular order π .

Theorem 6. *The problem of assortment optimization with customization under MMNL choice is an instance of cardinality constrained submodular order maximization.*

As a direct consequence of Theorem 6, we have a $0.5 - \epsilon$ approximation for any m . Table 2 provides approximation guarantees under more general constraint on the selection of products (budget and matroid). We establish submodular order property for MNL model and prove Theorem 6 in Online Appendix C.1. In fact, as a corollary of the result for matroid constraint, we obtain an approximation result for the problem of joint pricing and customization under MMNL choice (see Online Appendix C.1.1 for more details).

2.2.2. Results for Markov Model and Beyond. The order given by descending prices is not a submodular order for every choice model. The following example demonstrates this for the Markov model.

Example. Consider a ground set of four items indexed $i \in [4]$ in decreasing order of prices $r_1 = 8, r_2 = 4, r_3 = 4$, and $r_4 = 2$. Customer chooses in a markovian fashion starting at item 2 with a probability of one. If item 2 is not available, the customer transits to item j with probability $1/3$ for every $j \in \{1, 3, 4\}$. If j is also unavailable then the customer departs with probability of one. Consider the sets $B = \{1, 2\}$ and $A = \{1, 2, 3\}$. We have $f(A) = f(B) = 4$. However, if we add item 4 to these sets we get $f(4|A) = R(\{1, 3, 4\}) - 4 = 2/3$, whereas $f(4|B) = R(2) - R(2) = 0$.

It is not obvious (to us) if there is an alternative submodular order for Markov choice model. We give a procedure that extracts a *partial* submodular order for any given Markov choice model and show that all our algorithms for submodular order functions apply to assortment optimization in this model without any loss in guarantee. In fact, we show this more generally for any choice model that has the following structure.

Compatible choice models: Given a substitutable choice model ϕ , let S be an optimal unconstrained

Table 2. Results for Constrained Assortment Optimization

	MNL with customization	Compatible choice models (includes Markov model)
Cardinality	$\frac{\Omega(1)}{\log m} \rightarrow 0.5 - \epsilon$	$0.5 - \epsilon$
Budget/knapsack/capacity	$\frac{1}{m} \rightarrow 0.5 - \epsilon$	$\frac{1}{3} - \epsilon \rightarrow 0.5 - \epsilon$
Matroid (includes pricing and assortment)	$\frac{1}{m} \rightarrow 0.25$	$\frac{1}{d} \rightarrow 0.25$

Notes. New results are stated in bold and previous state-of-the-art provided to the left of each arrow (including results implied from but not explicitly stated in previous work). Note that m denotes the number of MNL models in the mixture and d is the matroid rank.

assortment on the ground set N . We say that ϕ is *compatible* if

$$R_\phi(A|C) \geq 0 \quad \forall A \subseteq S, C \subseteq N, \quad (2)$$

$$R_\phi(C|A) \leq R_\phi(C|B) \quad \forall B \subseteq A \subseteq S, C \subseteq N. \quad (3)$$

Compatibility is a structural property of optimal unconstrained assortments that gives sufficient conditions (in the absence of a submodular order) for translating algorithms (with guarantees) from submodular order maximization.

Theorem 7. *The family of Markov choice models is compatible. For any compatible choice model ϕ that admits a polynomial time algorithm for finding optimal unconstrained assortments, Algorithm 8 runs in polynomial time and matches the guarantee obtained by algorithms for submodular order functions under cardinality, budget, and matroid constraint (Table 2).*

Algorithm 8 is introduced in Section 3.3, and it builds on the algorithms for submodular order functions. We prove Theorem 7 in Online Appendix C.2. As an application of the previous theorem, we obtain the first non-trivial approximation for joint pricing and assortment optimization in the Markov model (see Online Appendix C.2 for a proof).

Corollary 1. *There is a 0.25 approximation for joint pricing and assortment optimization in the Markov choice model.*

2.2.2. Summary of Application to Assortment Optimization. The framework of submodular order functions provides a new algorithmic tool for constrained assortment optimization. Given a choice model (that is not MNL or Markov), we summarize the key steps to check if this tool can be applied to obtain efficient approximations.

- (i) Is the unconstrained assortment problem efficiently solvable? If not, is there a FPTAS?
- (ii) Is there a (strong or weak) submodular order? Specifically, is the descending order of revenues a submodular order?
- (iii) If the unconstrained assortment problem is efficiently solvable but a submodular order is not evident (or does not exist) is the choice model compatible?

Next, we discuss a very different application of submodular order maximization.

2.3. Application to Streaming Submodular Maximization

The streaming model: Consider a setting where the ground set N is ordered in some arbitrary manner, say $\{1, 2, \dots, n\}$. We have a monotone submodular objective f that we want to maximize subject to some constraint.

Because of the large size of N , elements can only be accessed sequentially in order, that is, to access element j we need to parse the data stream of elements from one to j . We also have a small working memory. Thus, we seek an algorithm that guarantees a good solution with very few passes (ideally, just one pass) over the data stream and requires very low memory (ideally, $\tilde{O}(k)$, where k is the size of a maximal feasible solution). This setting has a variety of applications in processing and summarizing massive data sets (Badanidiyuru et al. 2014). Standard algorithms for submodular maximization, such as the greedy algorithm for cardinality constraint, make $\Theta(kn)$ passes over the data stream given small working memory.

Previous work in streaming model: The first result in this model was given by Chakrabarti and Kale (2015), who gave a $\frac{1}{4p}$ approximation for maximization subject to intersection of p matroids. Badanidiyuru et al. (2014) gave a different algorithm with improved guarantee of $0.5 - \epsilon$ for cardinality constraint. Feldman et al. (2023) show that every streaming algorithm with guarantee better than $0.5 + \epsilon$ requires $O(\epsilon n/k^3)$ memory. Huang and Kakimura (2021) gave a $0.4 - \epsilon$ streaming algorithm for budget constrained optimization. Recently, Feldman et al. (2022b) gave improved approximation algorithms for matroid constraint. Chekuri et al. (2015) showed results for very general (p -matchoid) constraints and nonmonotone submodular functions. A more comprehensive review of related work can be found in Huang and Kakimura (2021) and Feldman et al. (2022b).

Connection to submodular order maximization: Consider a function f that is π -submodular order. Given the salience of order π , many of our algorithms for optimizing f are intentionally designed so that they parse elements in the order given by π . In fact, these algorithms can be efficiently implemented such that they parse the ground set exactly once and require very little memory. As a corollary, we recover the best-known guarantees for streaming submodular maximization in the cardinality constraint case, as well as constant factor approximation results for budget and matroid constraint.

Theorem 8. *For constrained maximization of a monotone submodular function f ,*

- (i) *Algorithm 1 gives a $(1 - \epsilon)0.5$ approximation in the streaming setting.*
- (ii) *Algorithm 3 gives a $\frac{1}{3} - \epsilon$ approximation algorithm in the streaming setting.*
- (iii) *Algorithm 9 gives a 0.25 approximation in the streaming setting.*

We include the proof in Online Appendix E. Algorithm 5, which is $0.5 - \epsilon$ approximate for budget constraint, requires working memory polynomial in n and is not a streaming algorithm. The main idea behind Theorem 8 is straightforward. Recall that a submodular

function satisfies the submodular order property on every permutation of the ground set. Therefore, an α approximate algorithm for submodular order functions that parses the ground set only once (in submodular order) and requires low memory is, by default, an α approximation algorithm for streaming submodular maximization. Although the connection is obvious in hindsight, it raises intriguing new questions. For example, the upper bound of 0.5 for submodular order functions (Theorem 3) is a consequence of their milder structure. In contrast, the upper bound of 0.5 for streaming maximization (Feldman et al. 2023) arises out of restrictions on memory. From a purely structural viewpoint, submodularity permits a stronger $(1 - 1/e)$ guarantee. *Is there a precise connection between less structure and memory limitations?*

3. Algorithms for Submodular Order Functions

In Example 1, we saw that in the absence of submodularity, iterative algorithms that augment the solution at each step (such as the family of greedy algorithms) may add elements that have large marginal value but *blind* the algorithm from picking up more “good” elements. Submodularity mutes this problem by guaranteeing that for any sets S and A , if $f(A) > f(S)$ then there exists an element $e \in A \setminus S$ such that, $f(e|S) \geq \frac{f(A) - f(S)}{|A|}$. In general, the absence of submodularity is severely limiting (Theorem 1). However, for functions with submodular order we have a lifeline. Consequently, all our algorithms treat the submodular order as salient and “process” elements in this order. Broadly speaking, we propose two types of algorithms. The first is inspired by greedy algorithms and the second by local search.

3.1. Threshold Based Augmentation in Submodular Order

Consider an instance of the cardinality constrained problem for a π submodular function. Fix an optimal solution and given set S , let OPT_S denote the subset of optimal products located to the right of S in the submodular order ($l_\pi(\text{OPT}_S) > r_\pi(S)$). Suppose we have an algorithm that maintains a feasible set at every iteration and let S_j denote this set at iteration j . Using weak submodular order property (and monotonicity), we can show that there exists e to the right of S_j such that

$$f(e|S_j) \geq \frac{f(S_j \cup \text{OPT}_{S_j}) - f(S_j)}{k}.$$

Now, if we greedily add an element to S_j , we may add an element very far down in the order. This may substantially shrink the set $\text{OPT}_{S_{j+1}}$ in the next iteration. To address this issue we consider a *paced* approach where

we add the first element e to the right of S_j such that

$$f(e|S_j) \geq \tau(N, f, k, S_j),$$

where $\tau(N, f, k, S_j)$ is a *threshold* that can depend on the instance N, f, k , and set S_j . In other words, we take marginal values into consideration by focusing only on elements that have sufficiently large value (but not the maximum value). From this filtered set of candidates we add the closest element to the right of S_j . Thus, minimizing the shrinkage in $\text{OPT}_{S_{j+1}}$.

The simplest possible threshold is a constant independent of the iteration. Quite surprisingly, for cardinality constraint, choosing a constant threshold leads to the best possible guarantee for maximizing (strong and weak) submodular order functions. Letting OPT denote the optimal value, any threshold close to $\frac{\text{OPT}}{2k}$ is a good choice. Although we do not know OPT a priori, trying a few values on a geometric grid brings us arbitrarily close. In Algorithm 2, let the ground set $N = \{1, 2, \dots, n\}$ be indexed in submodular order.

Algorithm 1 ($\frac{1}{2}$ for Cardinality)

Input: Cardinality k , error $\epsilon \in (0, 1)$;
Initialize $\tau = \frac{1}{k} \max_{e \in N} f(\{e\})$;
for $i \in \{1, 2, \dots, \lceil \log_{1+\epsilon} k \rceil\}$ **do** $S_i = \text{Threshold Add}(k, (1 + \epsilon)^{i-1} \tau)$;
end
Output: Best of $\{S_1, S_2, \dots, S_{\lceil \log_{1+\epsilon} k \rceil}\}$

Algorithm 2 (Threshold Add(k, τ))

Input: Cardinality k , threshold τ , N indexed in submodular order;
Initialize $S = \emptyset$;
for $i \in \{1, 2, \dots, n\}$ **and** $|S| < k$ **do** **if** $f(i|S) \geq \tau$ **then** $S \rightarrow S \cup \{i\}$;
;
end
Output: Set S of size at most k ;

Remarks. The idea of iteratively adding elements with marginal value above a threshold has a rich history in both submodular optimization and assortment optimization. Badanidiyuru and Vondrák (2014) gave a fast $(1 - 1/e) - \epsilon$ approximation for constrained maximization of submodular functions using adaptive thresholds. Badanidiyuru et al. (2014) proposed an adaptive threshold algorithm for cardinality constraint in the streaming model. In the assortment optimization literature, Désir et al. (2020) introduce a constant threshold algorithm for cardinality and budget constrained assortment optimization in the Markov model.

Generalizing to budget constraint: Algorithm 3 presents a natural generalization with a threshold on the “bang-per-buck,” that is, *marginal value per unit budget*. Formally, we filter out elements e such that $\frac{f(e|S_j)}{b_e} < \tau(N, f, B)$. We show a guarantee of $1/3$ for Algorithm 3, short of the upper bound of 0.5 implied by Theorem 3.

A standard technique in submodular maximization (and beyond) to obtain the best possible guarantees under budget constraint is *partial enumeration*. For example, Sviridenko (2004) considers all possible sets of size 3 as a starting point for a greedy algorithm that picks an item with best bang-per-buck at each step thereafter. Starting with an initial set X is equivalent to changing the objective function to $f(\cdot | X)$. In case of submodular order functions $f(\cdot | X)$ will, in general, not have a weak submodular order even for a small set X . To preserve submodular order, we may restrict enumeration to starting sets that are concentrated early in the order. However, this limited enumeration appears to be ineffective. Instead, we propose an algorithm that starts with an empty set and parses elements in submodular order but enumerates over a small set of high budget elements which get “special attention” via the Final Add subroutine. It should be noted that unlike Algorithm 3, which only adds elements, Algorithm 5 may discard elements that were added in previous iterations via the Final Add subroutine.

Algorithm 3 ($\frac{1}{3}$ for Budget)

Input: Budget B , error $\epsilon \in (0, 1)$;
Initialize $\tau = \frac{1}{B} \max_{e \in N} f(\{e\})$;
for $i \in \{1, 2, \dots, \lceil \log_{1+\epsilon} n \rceil\}$ **do**
 $S_i = \text{B-Th. Add}(B, (1 + \epsilon)^{i-1} \tau)$;
end
Output: Best of all *singletons* and sets $\{S_1, S_2, \dots, S_{\lceil \log_{1+\epsilon} n \rceil}\}$

Algorithm 4 (B-Th Add (B, τ))

Input: Budget B , threshold τ , N indexed in submodular order;
Initialize $S = \emptyset$;
for $i \in \{1, 2, \dots, n\}$ **and** $b(S) < B$ **do**
 if $b_i < B - b(S)$ **and** $\frac{f(i|S)}{b_i} \geq \tau$ **then**
 $S \rightarrow S \cup \{i\}$;
end
Output: Feasible set S ;

Algorithm 5 (0.5 for Budget)

Input: Budget B , error $\epsilon \in (0, 1)$, N indexed in submodular order;
Initialize collection $E = \{X \subseteq N | b(X) \leq B, |X| \leq 1/\epsilon\}$;
for $X \in E$ **do**
 Filter ground set $N \rightarrow N \setminus \{e | b_e > \min_{i \in X} b_i\}$;
 Initialize $\tau = \frac{1}{B} \max_{e \in N} f(\{e\})$;
 for $i \in \{1, 2, \dots, \lceil \log_{1+\epsilon} |N| \rceil\}$ **do**
 Initialize $S_i = \emptyset$;
 for $j \in \{1, 2, \dots, |N|\}$ **and** $b(S_i) < B$ **do**
 if $\frac{f(j|S_i)}{b_j} \geq (1 + \epsilon)^{i-1} \tau$ **then**
 if $b_j + b(S_i) \leq B$ **then**
 $S_i \rightarrow S_i \cup \{j\}$
 else
 Update $S_i \rightarrow \text{Final Add}(B, \epsilon, S_i \cup \{j\})$;

```

    | | | | Goto next i;
    | | | | end
    | | | | end
    | | | | end
    | | | | end
    | | | | Let  $S(X) = \text{Best of all feasible sets } S_i$ ;
    | | | | end
    | | | | Output: Best of all sets  $\{S(X) | X \in E\}$ 

```

Algorithm 6 (Final Add)

Input: Budget B, ϵ , set S ;
while $b(S) > B$ **do**
 Remove an element i with budget $b_i < \epsilon B$ from S ;
 End loop if no such element exists;
end
Output: S ;

The following example illustrates why threshold based algorithms fail under matroid constraints.

Example. Consider a matroid with rank $2^n - 1$ on the ground set $\{1, \dots, 2^{n+1} - 2\}$, indexed in submodular order. Let R denote the set $\{1, \dots, 2^n - 1\}$, that is, the first half of the ground set. Let P denote the other half. Partition R into ordered sets $\{R_1, \dots, R_n\}$ such that R_1 is the set $[2^{n-1}]$ of the first 2^{n-1} elements, R_2 of the next 2^{n-2} elements, and so on. Therefore, $|R_i| = 2^{n-i}$ for every $i \in [n]$. Partition P into ordered sets $\{P_1, \dots, P_n\}$ so that P_i is the set $\{2^n - 1 + 2^{i-1}, 2^n - 2 + 2^i\}$ of 2^{i-1} elements. Observe that $|R_i| = |P_{n-i+1}|$. We define independent sets in the matroid so that the set $R_i \cup (\cup_{j=n-i+1}^n P_j)$ is independent for every $i \in [n]$. Therefore, P is independent and so is the set R_i (and P_i) for every i . Additionally, let the rank of the set $R_i \cup R_{i+j}$ equal $|R_i|$ for every $j \geq 1$. Finally, define a modular function f so that P is optimal and $f(P) = 2^n - 1$. Let $f(P_i) = f(P)/n$ and $f(e) = f(P_i) / |P_i|$, for every $i \in [n]$ and $e \in P_i$. Similarly, let $f(R_i) = f(P_{n-i+1}) = f(P)/n$ and $f(e) = f(R_i) / |R_i|$, for every $i \in [n]$ and $e \in R_i$. Now, for any given threshold τ , the algorithm that parses the elements in order and selects every element with marginal value exceeding τ (while maintaining independence) will pick a set with value exactly $2f(P)/n$ in this instance (translating to a factor $2/\log n$ for ground set of size n).

3.2. Local Search Along Submodular Order

To address the challenges with matroid constraint, we introduce an algorithm inspired by local search. Given a feasible set S , a local search algorithm tries to make “small” changes to the set in order to improve function value. Often, this is done via *swap operations* where a set X of elements is added to S and a subset Y (can be empty) is removed such that $f(S \cup X \setminus Y) \geq (1 + \epsilon)f(S)$, for some $\epsilon > 0$, and the new set $S \cup X \setminus Y$ is feasible.¹ The algorithm terminates at local maxima and for submodular functions the local maxima are within a constant factor of optimal Feige et al. (2011). As illustrated

in Example 1, functions with submodular order have poor local maxima. Therefore, we perform a *directed* local search where we consider elements for swap operations one-by-one in the submodular order. Consider the following algorithm based on this idea,

Parse elements in submodular order and add j to current set S if there exists an element i (could be $\emptyset$) such that $S' := (S \setminus \{i\}) \cup \{j\} \in \mathcal{I}$ and $f(S') \geq (1 + \epsilon)f(S)$.

At first, one may prefer to set a small value of ϵ . This would entail slower convergence but is expected to yield stronger approximation guarantee. However, unlike standard local search, the algorithm above parses each element exactly once. It turns out that unless ϵ is large enough, a single pass over the elements may end with a poor chain of swaps $i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_k$. This is demonstrated in the next example.

Example. Consider the independent set $S = \{1, \dots, m\}$ with $f(S) = 1$ at iteration $m + 1$. Let $\{m, m + 1\}$ form a circuit and suppose $m + 1$ meets the criteria for a swap when $1/\epsilon = O(m^2)$ (quite small). Therefore, we have $S = [m - 1] \cup \{m + 1\}$ after iteration $m + 1$ and let $f(S) = 1 + \epsilon$. In subsequent iteration, suppose $m + 2$ makes a circuit with element $2 \in S$ and let $f(S \cup \{m + 2\} \setminus \{2\}) = f(S)$. Thus, $m + 2$ does not meet the criteria for a swap. However, $f(\{m, m + 2\}) = 2$. In the next iteration, we swap out $m + 1$ in favor of $m + 3$ and for the resulting set let $f(S) = 1 + 2\epsilon$. Next, we find that element $m + 4$ does not meet the criteria for a swap but $f(\{m, m + 2, m + 4\}) = 3$. Inductively, after $2m$ such iterations we have an instance such that the local search solution value is $1 + m\epsilon \approx 1 + 1/m$, while the optimal value is m .

At a high level, we prevent such poor chains of swaps from occurring by constantly adjusting the swap criteria so that the k -th element in a chain $i_1 \rightarrow \dots \rightarrow i_k$ of swaps has marginal value greater than equal to the sum of the marginal values of all previous elements in the chain.

Algorithm 7 ($\frac{1}{\epsilon}$ for Matroid)

Input: Independence system $\mathcal{I}$, N indexed in submodular order;

Initialize $S, R = \emptyset$ and values $v_j = 0$ for all $j \in [n]$;

for $j \in \{1, 2, \dots, n\}$ **do**

if $S \cup \{j\} \in \mathcal{I}$ **then** initialize $v_j = f(j|S \cup R)$ and update $S \rightarrow S \cup \{j\}$;

else

 Find circuit C in $S \cup \{j\}$ and define $i^* = \arg \min_{i \in C \setminus \{j\}} v_i$ and $v_C = v_{i^*}$;

if $f(j|S \cup R) > v_C$ **then**

$v_j \rightarrow v_C + f(j|S \cup R)$;

$S \rightarrow (S \setminus \{i^*\}) \cup \{j\}$ and $R \rightarrow R \cup \{i^*\}$;

end

end

end

Output: Independent set S

Remark. For functions with strong submodular order, we can use marginals $f(j|S)$ instead of $f(j|S \cup R)$ without changing the approximation guarantee. The modified algorithm (Algorithm 9), and its performance analysis is included in Online Appendix E. The fact that set $S \cup R$ grows monotonically in Algorithm 7 is critical in application to assortment optimization of compatible choice models. Conversely, using $f(j|S)$ makes it possible to implement Algorithm 9 in the streaming setting where memory is small ($\tilde{O}(d)$).

3.3. Algorithms for Constrained Assortment Optimization

In this section, we translate the algorithms from submodular order maximization to constrained assortment optimization. Recall that the revenue objective in assortment optimization can be a nonmonotone function. Therefore, we transform the objective to $f_\phi(S) = \max_{X \subseteq S} R_\phi(S)$, which is monotone and subadditive for choice models that are substitutable. In an ideal scenario, we have a choice model ϕ , where the unconstrained assortment optimization can be solved efficiently (or admits a FPTAS) and f_ϕ has a submodular order. If a submodular order is not evident, or does not exist, but the choice model is compatible (a property of optimal unconstrained assortments), we propose a new framework.

Our framework for compatible choice models builds on the algorithms for submodular order functions and sequentially constructs an order over the ground set, satisfying a relaxed notion of the submodular order property. To define the framework, we need some notation. Let $\mathcal{A}$ represent an algorithm for constrained submodular order maximization (out of Algorithms 1, 3, 5, and 7). An instance of the assortment optimization problem is given by set N , function f_ϕ , and constraint $\mathcal{F}$. Let $\Gamma_{\mathcal{A}}(N, f_\phi, \mathcal{F})$ define the set of parameter settings that $\mathcal{A}$ enumerates. We explicitly define this set for each algorithm later. For brevity, we write $\Gamma_{\mathcal{A}}(N, f_\phi, \mathcal{F})$ simply as $\Gamma_{\mathcal{A}}$. Given a parameter setting $\gamma \in \Gamma_{\mathcal{A}}$, let

$$(S, R) := \mathcal{A}_\gamma(f, N, \pi, \mathcal{F}),$$

denote the output of $\mathcal{A}$ when ground set N is ordered according to π . The set S is the feasible solution generated by $\mathcal{A}_\gamma$. R is a minimal set such that every query made by $\mathcal{A}_\gamma$ during its execution is a marginal value $f(\cdot|X)$ for some $X \subseteq S \cup R$. Finally, let $U_\phi(X)$ denote an optimal unconstrained assortment on ground set X . Observe that $f_\phi(X)$ is the expected revenue of $U_\phi(X)$. Before stating our new framework, we explicitly define $\Gamma_{\mathcal{A}}$ and R for each algorithm.

$\Gamma_{\mathcal{A}}$ and R for Algorithms 1, 3, and 5: $\Gamma_{\mathcal{A}}$ is the set of all threshold values τ tried in the algorithm. For Algorithm 5, the set also includes sets X of high budget elements used to filter the ground set. In Algorithms 1 and 3 marginal values are evaluated only with respect

to subsets of the output S . So $R = \emptyset$ for every parameter setting. In Algorithm 5, R is the set of elements discarded by the Final Add subroutine (Algorithm 6).

$\Gamma_{\mathcal{A}}$ and R for Algorithm 7: There is no enumeration in Algorithm 7 so $\Gamma_{\mathcal{A}}$ is a singleton set (the **for** loop executes once). R is the set of all elements that were picked by the algorithm and then swapped out.

Algorithm 8 (Constrained Assortment Optimization for Compatible Choice Models)

Input: Algorithm $\mathcal{A}$, set N , constraint $\mathcal{F}$, oracles for f_{ϕ}, U_{ϕ} ;

```

for  $\gamma \in \Gamma_{\mathcal{A}}$  do
  Initialize  $\hat{N} = U_{\phi}(N)$ ,  $M = \emptyset$  and arbitrary order
   $\pi_{\hat{N}} : \hat{N} \rightarrow \{1, \dots, \hat{N}\}$ ;
  while  $\hat{N} \setminus M \neq \emptyset$  do
     $(S, R) = \mathcal{A}_{\gamma}(f_{\phi}, \hat{N}, \pi_{\hat{N}}, \mathcal{F})$ ;
     $M = S \cup R$ ;
     $N \rightarrow (N \setminus \hat{N}) \cup M$ ;
     $\hat{N} \rightarrow U_{\phi}(N) \cup M$ ;
    Update  $\pi_{\hat{N}}$  by adding elements in  $\hat{N} \setminus M$  last in
    the order;
  end
   $S_{\gamma} := S$ 
end

```

Output: Best of $\{S_{\gamma}\}_{\gamma \in \Gamma_{\mathcal{A}}}$

At a high level, given a constrained assortment optimization problem without a submodular order, the framework sequentially constructs a (partial) submodular order. It starts with the unconstrained optimal assortment $U_{\phi}(N)$, denoted as N_1 for brevity. For a compatible choice model, we show that every order π that places elements in N_1 before all other elements, defines a partial submodular order in the following sense:

$$f(C|A) \leq f(C|B) \quad \forall B \subseteq A \subseteq N_1, C \subseteq N.$$

Given this insight and the fact that our algorithms for submodular order functions parse the ground set in submodular order, the framework now executes $\mathcal{A}$ with the truncated ground set N_1 . Let $M_1 = S \cup R$ denote the output of $\mathcal{A}$ after this first phase. The set $N_1 \setminus M_1$ of elements that are parsed and rejected by $\mathcal{A}$ are now discarded and we update $N \rightarrow (N \setminus N_1) \cup M_1$, which is the set of remaining elements. Having parsed elements in N_1 , the framework now augments the (partial) submodular order by resolving for an unconstrained optimal assortment on the new ground set N . The new set $U_{\phi}(N)$ may now include elements that were not previously parsed by $\mathcal{A}$. Let $N_2 = U_{\phi}(N) \setminus M_1$ denote this new set of elements. The framework adds N_2 to the end of the partial submodular order (to the right of N_1) and executes $\mathcal{A}$ now on the ground set $M \cup N_2$. Repeating this process several times, new elements are incrementally passed into $\mathcal{A}$ in a growing partial order π . The

algorithm terminates when no new elements appear in the unconstrained assortment (after at most n phases).

To summarize, Algorithm 8 uses the algorithms for submodular order maximization to solve a constrained assortment optimization when it is not evident that the choice model has a submodular order. This framework can be used to solve constrained assortment optimization for any choice model that admits an efficient algorithm for solving the unconstrained problem. However, the approximation guarantees may not hold for non-compatible choice models. The analysis of Algorithm 8 is presented in Online Appendix C.2 (and Online Appendix D).

4. Approximation Guarantees

In this section, we analyze the performance of our algorithms for maximizing functions with (known) submodular order. In Online Appendix B, we prove an upper bound (Theorem 3) on the approximation guarantee of efficient algorithms. For submodular functions, given a set A and partition $\{O, E\}$ of A , we have

$$f(A) = f(E) + f(O|E) \leq f(E) + \sum_{i \in O} f(i|E).$$

The previous inequality is at the heart of proving tight guarantees for maximizing monotone submodular functions. For functions with submodular order this property need not hold unless all elements of O are located entirely to the right of E in the submodular order. In the following, we show stronger upper bounds that play a crucial role in the analysis of all our algorithms. First, we define the notion of interleaved partitions.

Interleaved partitions: Given a set A and an order π over elements, an interleaved partition

$$\{O_1, E_1, O_2, \dots, O_m, E_m\}$$

of A is given by sets $\{O_{\ell}\}_{\ell \in [m]}$ and $\{E_{\ell}\}_{\ell \in [m]}$ that alternate and never cross each other in the order π . Formally, for every $\ell \geq 1$, we have $r_{\pi}(O_{\ell}) < l_{\pi}(E_{\ell}) \leq r_{\pi}(E_{\ell}) < r_{\pi}(O_{\ell+1})$. To assist the reader, we note that letters O and E signify odd and even numbered sets in the partition. As noted previously, for functions with submodular order the upper bound $f(A) \leq f(\cup_{\ell \in [m]} E_{\ell}) + f(\cup_{\ell \in [m]} O_{\ell} | \cup_{\ell \in [m]} E_{\ell})$ does not always hold. Instead, we establish a family of (incomparable) upper bounds parameterized by permutation $\sigma : [m] \rightarrow [m]$. The flexibility provided by permutation σ is particularly helpful in analyzing Algorithms 5 and 7, where elements may be swapped out.

The following set unions will be used extensively in the lemmas that follow. Given permutation σ , let

$$E(j) = \bigcup_{\ell \leq j} E_{\ell} \text{ and } O_{\sigma}(j) = \bigcup_{\ell \mid \sigma(\ell) \geq \sigma(j)} O_{\ell},$$

with $E(0) := \emptyset$ and $O_{\sigma}(m+1) := \emptyset$. When $\sigma(\ell) = \ell$, $\forall \ell \in [m]$, we use the shorthand $O(j) = \cup_{\ell \geq j} O_{\ell}$. In this

notation, $E(m) = \cup_{\ell \in [m]} E_\ell$, $O(1) = \cup_{\ell \in [m]} O_\ell$. Thus, $A = O(1) \cup E(m)$. Next, for every $j \in [m]$ define

$$L_\sigma(j) = \bigcup_{\ell | \ell < j, O_\ell \subseteq O_\sigma(j)} O_\ell,$$

which is the union over sets O_ℓ that are to the left of O_j in submodular order and to the right of O_j in permutation σ . When $\sigma(\ell) = \ell$, $\forall \ell \in [m]$, we have $L_{j,\sigma} = \emptyset$ for every $j \in [m]$.

Lemma 1. *Given a set function f with weak submodular order π , set A with interleaved partition $\{O_\ell, E_\ell\}_{\ell=1}^m$, and a permutation $\sigma : [m] \rightarrow [m]$, we have*

$$f(A) \leq f(E(m)) + \sum_{\ell \in [m]} f(O_\ell | L_\sigma(\ell) \cup E(\ell - 1)).$$

Proof. The following inequalities are crucial for the proof:

$$\begin{aligned} f(O_\sigma(\ell) \cup E(m)) &\leq f(O_\ell | L_\sigma(\ell) \cup E(\ell - 1)) \\ &\quad + f(O_\sigma(\ell) \cup E(m) \setminus O_\ell) \quad \forall \ell \in [m]. \end{aligned} \quad (4)$$

Before we establish (4), observe that by summing these inequalities for $\ell \in [m]$, we get

$$\begin{aligned} \sum_{\ell \in [m]} f(O_\sigma(\ell) \cup E(m)) - \sum_{\ell \in [m]} f(O_\sigma(\ell) \cup E(m) \setminus O_\ell) \\ \leq \sum_{\ell \in [m]} f(O_\ell | L_\sigma(\ell) \cup E(\ell - 1)), \\ f(O(1) \cup E(m)) - f(E(m)) \leq \sum_{\ell \in [m]} f(O_\ell | L_\sigma(\ell) \cup E(\ell - 1)), \\ f(A) - f(E(m)) \leq \sum_{\ell \in [m]} f(O_\ell | L_\sigma(\ell) \cup E(\ell - 1)), \end{aligned}$$

as desired. It remains to show (4). Consider arbitrary $\ell \in [m]$ and notice that sets $B := L_\sigma(\ell) \cup E(\ell - 1)$ and $A := L_\sigma(\ell) \cup E(\ell - 1) \cup O_\ell$ are π -nested. Moreover, the set $C := O_\sigma(\ell) \cup E(m) \setminus A$ lies entirely to the right of A . From the weak submodular order property, we have $f(C|A) \leq f(C|B)$. Consequently,

$$\begin{aligned} f(O_\sigma(\ell) \cup E(m)) &= f(A) + f(C|A) \\ &\leq f(B) + f(O_\ell|B) + f(C|B) \\ &= f(O_\ell|B) + f(B \cup C) \\ &= f(O_\ell | L_\sigma(\ell) \cup E(\ell - 1)) \\ &\quad + f(O_\sigma(\ell) \cup E(m) \setminus O_\ell). \quad \square \end{aligned}$$

Remark. The following corollaries of the general bound in Lemma 1 suffice for all our analyses. In the first corollary, we choose σ to be identity. In the second, we let σ be the reverse order permutation.

Corollary 2. *Given a set function f with weak submodular order π , set A with interleaved partition $\{O_\ell, E_\ell\}_{\ell=1}^m$, and the permutation $\sigma(\ell) = \ell$ for every $\ell \in [m]$, we have*

$$f(A) \leq f\left(E(m) + \sum_{\ell \in [m]} f(O_\ell | E(\ell - 1))\right).$$

Corollary 3. *Consider a set function f with weak submodular order π and set A with interleaved partition $\{O_\ell, E_\ell\}_{\ell=1}^m$. Let $L_\ell = \{e \in A | \pi(e) < \pi(E_\ell)\}$. Then,*

- (i) $f(A) \leq f(E(m)) + \sum_{\ell \in [m]} f(O_\ell | E(\ell - 1) \cup O(1) \setminus O(\ell))$,
- (ii) $\sum_{\ell \in [m]} f(E_\ell | L_\ell) \leq f(E(m))$.

Proof. Applying Lemma 1 with $\sigma[\ell] = m - \ell + 1$ for $\ell \in [m]$, we have

$$f(A) \leq f(E(m)) + \sum_{\ell \in [m]} f(O_\ell | E(\ell - 1) \cup O(1) \setminus O(\ell)).$$

To complete the proof, we use the decomposition

$$f(A) = \sum_{\ell} [f(E_\ell | L_\ell) + f(O_\ell | E(\ell - 1) \cup O(1) \setminus O(\ell))]. \quad \square$$

4.1. Cardinality Constraint

Proof of Theorem 2. We start by calculating the number of queries made by Algorithm 1. The Threshold Add subroutine makes at most n queries. Algorithm 1 calls the Threshold Add subroutine $\lceil \log_{1+\epsilon} k \rceil$ times. This results in $O(n \log_{1+\epsilon} k) = O(\frac{n}{\epsilon} \log k)$ queries.

Next, we establish the approximation guarantee. Let $\tau_i = \tau(1 + \epsilon)^{i-1}$. We use OPT to denote both the optimal solution and function value. For $\text{OPT} \leq 2 \max_{e \in N} f(\{e\})$, we have $f(S_{\lceil \log_{1+\epsilon} k \rceil}) \geq (1 - \epsilon) 0.5 \text{OPT}$. Therefore, from here on we let $\text{OPT} > 2 \max_{e \in N} f(\{e\})$. Consequently, there exists an i such that

$$(1 - \epsilon) \frac{\text{OPT}}{2k} \leq \tau_i \leq \frac{\text{OPT}}{2k}.$$

We show that $f(S_i) \geq (1 - \epsilon) 0.5 \text{OPT}$. For convenience, we omit i from the subscript and write S_i simply as S and τ_i as τ .

Let k' denote the cardinality of set S . By definition of Threshold Add,

$$f(S) \geq k' \tau.$$

When $k' = k$, this gives us $f(S) \geq (1 - \epsilon) 0.5 \text{OPT}$. Therefore, let $k' < k$. From monotonicity of the function, we have $\text{OPT} \leq f(\text{OPT} \cup S)$. Therefore, consider $\text{OPT} \cup S$ and its interleaved partition

$$\{O_1, \{s_1\}, O_2, \{s_2\}, \dots, \{s_{k'}\}, O_{k'+1}\},$$

where s_j denotes the j th element added to S . Set O_{j+1} contains all elements in OPT between s_j and s_{j+1} , that is,

$\pi(s_j) < l_\pi(O_{j+1})$ and $r_\pi(O_j) < \pi(s_j)$ for every $j \in [k']$. Some sets may be empty. Applying Corollary 2 on $\text{OPT} \cup S$ with $E_j = s_j$ for $j \in [k']$, we have

$$\begin{aligned} \text{OPT} &\leq f(E(k')) + \sum_{\ell \in [k']} f(O_\ell | E(\ell - 1)), \\ &= f(S) + \sum_{\ell \in [k']} f(O_\ell | E(\ell - 1)). \end{aligned} \quad (5)$$

Using weak submodular order, we have for every $\ell \geq 1$,

$$f(O_\ell | E(\ell - 1)) \leq \sum_{e \in O_\ell} f(e | E(\ell - 1)) \leq \tau |O_\ell|,$$

where the second inequality follows the fact that for $k' < k$, elements in $\text{OPT} \setminus S$ did not meet the criteria in Threshold Add (Algorithm 2). Plugging this into (5) and using the upper bound $\tau \leq 0.5 \text{OPT}/k$, we get

$$\text{OPT} \leq f(S) + k\tau \leq f(S) + 0.5 \text{OPT}.$$

This completes the proof. Now, suppose we have a noisy function oracle $\hat{f}$, such that $(1 - \delta)f(S) \leq \hat{f}(S) \leq (1 + \delta)f(S) \forall S \subseteq N$. Observe that $f(e | S) \leq \frac{1}{1 - \delta} \hat{f}(S + e) - \frac{1}{1 + \delta} \hat{f}(S) \leq \hat{f}(e | S) + \frac{2\delta}{1 - \delta} f(S + e)$. For $k' = k$, we have $f(S) \geq \frac{1}{1 + \delta} \hat{f}(S) \geq \frac{1}{1 + \delta} k' \tau \geq \frac{1}{1 + \delta} (1 - \epsilon) 0.5 \text{OPT}$. For $k' < k$, we have

$$\begin{aligned} f(e | E(\ell - 1)) &\leq \hat{f}(e | E(\ell - 1)) + 2 \frac{\delta}{1 - \delta} f(e \cup E(\ell - 1)) \\ &< \tau + 2 \frac{\delta}{1 - \delta} \text{OPT} \quad \forall e \in O_\ell. \end{aligned}$$

Therefore, $f(S) \geq (0.5 - \frac{2k\delta}{1 - \delta}) \text{OPT}$. Overall, in the presence of a noisy oracle, we have approximation guarantee $\min\{\frac{0.5}{1 + \delta}(1 - \epsilon), 0.5 - \frac{2k\delta}{1 - \delta}\} = (1 - \epsilon)(1 - O(\frac{n\delta}{1 - \delta}))0.5$. $\square$

4.2. Budget Constraint

Proof of Theorem 4(i). The query complexity of Algorithm 3 is identical to Algorithm 1. To establish the approximation guarantee, observe that we ignore elements $i \in N$ with $b_i > B$. Also, let $\sum_{i \in N} b_i > B$ (otherwise picking the ground set is optimal). Let $\tau_i = \tau(1 + \epsilon)^{i-1}$. For $\text{OPT} \leq 2 \max_{e \in N} f(\{e\})$, the final solution has value at least 0.5OPT due to the singleton $\arg \max_{e \in N} f(\{e\})$. Therefore, from here on, we let $\text{OPT} > 2 \max_{e \in N} f(\{e\})$. Consequently, there exists an i such that

$$(1 - \epsilon) \frac{2 \text{OPT}}{3B} \leq \tau_i \leq \frac{2 \text{OPT}}{3B}.$$

We will show that $f(S_i) \geq \frac{1 - \epsilon}{3} \text{OPT}$. Let B_i denote the total budget used by set S_i . By definition of B-Threshold Add,

$$f(S_i) \geq B_i \tau_i.$$

Thus, for $B_i \geq \frac{1}{2}B$, we have $f(S_i) \geq \frac{1 - \epsilon}{3} \text{OPT}$. Therefore, let $B_i < \frac{1}{2}B$. Similar to the proof of Theorem 2, consider

the set $\text{OPT} \cup S_i$ and its interleaved partition:

$$\{O_1, \{s_1\}, O_2, \{s_2\}, \dots, \{s_{k-1}\}, O_k \cup \{i^*\}, \dots, \{s_{k_i}\}, O_{k_i+1}\},$$

where s_j is the j th element added to S_i . The main difference is the possible existence of element i^* (in OPT) that meets the threshold requirement but cannot be added to S_i due to the budget constraint. There can only be one such element as $B_i < \frac{1}{2}B$. Applying Corollary 2, we have

$$\begin{aligned} f(\text{OPT} \cup S_i) &\leq f(S_i) + f(\{i^*\} | E(\kappa - 1)) \\ &\quad + \sum_{\ell \in [k_i+1]} f(O_\ell | E(\ell - 1)). \end{aligned}$$

We consider two cases based on existence of i^* . Let ALG denote the value of solution generated by the algorithm.

Case I: i^* does not exist. Let B_j denote the total budget of set O_j . Using the upper bound $f(O_j | E(j - 1)) \leq \tau_i B_j$ for every $j \in [k_i + 1]$, we have

$$\text{OPT} \leq f(S_i) + \tau_i \sum_{j \in [k_i+1]} B_j = f(S_i) + \frac{2}{3} \text{OPT}.$$

Thus, $\text{ALG} \geq f(S_i) \geq \frac{1}{3} \text{OPT}$.

Case II: i^* exists. Then, $f(\{i^*\} | E(\kappa - 1)) \geq \tau_i b_{i^*}$ and $b(S_i) + b_{i^*} > B$. Therefore,

$$f(S_i) + f(\{i^*\} | E(\kappa - 1)) \geq \tau_i B \geq (1 - \epsilon) \frac{2}{3} \text{OPT}. \quad (6)$$

Now, the algorithm outputs the best of S_i and all singletons. Therefore,

$$2 \text{ALG} \geq f(S_i) + f(i^*) \geq f(S_i) + f(\{i^*\} | E(\kappa - 1)),$$

here second inequality follows from submodular order property. Using (6) completes the proof.

Given a noisy function oracle f with multiplicative error $(1 \pm \delta)$, we have $\hat{f}(S_i) \geq B_i \tau_i$. For $B_i \geq 0.5B$, we get $f(S_i) \geq \frac{1 - \epsilon}{3(1 + \delta)} \text{OPT}$. For $B_i < 0.5B$ and when i^* does not exist, we have $f(O_j | E(j - 1)) \leq \tau_i B_j + 2 \frac{|O_j| \delta}{1 - \delta} \text{OPT}$ for every $j \in [k_i + 1]$. Thus, $f(S_i) \geq (1 - \frac{6m\delta}{1 - \delta}) \frac{\text{OPT}}{3}$. Finally, if i^* exists, we have $\hat{f}(S_i) + \hat{f}(\{i^*\}) \geq \tau_i B$, and therefore, $\text{ALG} \geq (1 - O(\delta)) \frac{\text{OPT}}{3}$. Overall, in the presence of a noisy oracle, we have approximation guarantee $\frac{\min\{(1 - O(\delta))(1 - \epsilon), 1 - \frac{6m\delta}{1 - \delta}\}}{3} = (1 - \epsilon)(1 - O(\frac{n\delta}{1 - \delta})) \frac{1}{3}$. $\square$

Proof of Theorem 4(ii). For each X the algorithm makes $O(\frac{n}{\epsilon} \log n)$ queries, and there are $O(n^{1/\epsilon})$ possible sets X leading to $O(\frac{1}{\epsilon} n^{1+\frac{1}{\epsilon}} \log n)$ queries in total.

To show the guarantee we focus on set $S(X)$ when $X \subseteq \text{OPT}$ contains the $1/\epsilon$ largest budget elements in OPT (or all of OPT if its cardinality is small). For this X , the budget b_e required by any $e \in \text{OPT} \setminus X$ is strictly smaller than ϵB , otherwise $b(X) > B$.

Consider $\tau_i \in [(1 - \epsilon) \frac{\text{OPT}}{2B}, \frac{\text{OPT}}{2B}]$ and let S_i denote the solution returned by the algorithm with X and threshold

τ_i . We show that $f(S_i) \geq (0.5 - \epsilon) \text{OPT}$. The analysis is split in two cases based on how the algorithm terminates.

Case I: Final Add is not invoked. In this case, every element not in S_i fails the threshold requirement and the algorithm does not remove elements after they are chosen. Similar to the analysis of Algorithm 1, we have an interleaved partition

$$\{O_1, \{s_1\}, O_2, \{s_2\}, \dots, \{s_{k_i}\}, O_{k_i+1}\},$$

of $\text{OPT} \cup S_i$ such that s_j is the j th element added to S_i . Using Corollary 2, we have

$$\text{OPT} \leq f(S_i) + \sum_{j=1}^{k_i+1} f(O_j | E(j-1)).$$

Now, $f(O_j | E(j-1)) \leq \tau_i |O_j|$, $\forall j \in [k_i+1]$. Thus, $\text{ALG} \geq f(S_i) \geq 0.5 \text{OPT}$.

Case II: Final Add is invoked for some element j . We claim that the set S_i output by Final Add is such that (i) $B \geq b(S_i) \geq (1 - \epsilon)B$ and (ii) $f(S_i) \geq \tau_i b(S_i)$. Using (i) and (ii), we have for $\epsilon \in [0, 1]$,

$$f(S_i) \geq 0.5(1 - \epsilon)^2 \text{OPT} \geq (0.5 - \epsilon) \text{OPT}.$$

It remains to show (i) and (ii). Let S_{in} denote the set that is input to Final Add and let $X_i = \{e | b_e \geq \epsilon B, e \in S_{in}\}$. Observe that $X_i \subseteq X$ because every element outside X with budget exceeding ϵB is discarded in the beginning. Therefore, $b(X_i) \leq B$ and the set S_i returned by Final Add is feasible (and contains X_i). To see the lower bound on $b(S_i)$, let t denote the last element removed from S_{in} by Final Add. We have, $b_t < \epsilon B$ and $b_t + b(S_i) > B$. Thus, $b(S_i) \geq B - \epsilon B$.

To show (ii) we use Corollary 3(ii). For simplicity, let $\{1, 2, \dots, s\}$ denote the elements of S_i in submodular order. Recall that $S_i \subseteq S_{in}$. Using Corollary 3(ii) with $A = S_{in}$ and sets $E_k = \{k\}$ for $k \in [s]$, we have

$$\sum_{k \in [s]} f(k | L_k) \leq f(S_i),$$

where L_k denotes the set of all elements in S_{in} chosen by the algorithm prior to k . From the threshold requirement, we have $f(k | L_k) \geq \tau_i b_k$, $\forall k \in [s]$. Thus, $f(S_i) \geq \tau_i b(S_i)$.

In case of a noisy oracle, similar to previous analyses it can be verified that the approximation guarantee is reduced by a multiplicative factor $(1 - O(\frac{n\delta}{1-\delta}))$. $\square$

4.3. Matroid Constraint

Lemma 2. Given a matroid $\mathcal{M}$ and an independent set A , define $\mathcal{I}_A = \{B | A \cup B \in \mathcal{I}, B \subseteq N \setminus A\}$. For every independent set A' such that $r(A') = r(A \cup A') = r(A)$, we have $\mathcal{I}_A = \mathcal{I}_{A'}$.

Proof. Consider an arbitrary set $B \in \mathcal{I}_A$. It suffices to show that $B \in \mathcal{I}_{A'}$. There are many ways to show this, in the spirit of submodularity we will prove this by using the submodularity of rank function $r(\cdot)$.

Note that $r(A \cup B) = |A| + |B| = |A'| + |B|$ and consider the sets $A' \cup B$ and $A \cup A'$. We have from submodularity,

$$\begin{aligned} r(A' \cup A \cup B) + r(A') &\leq r(A' \cup B) + r(A \cup A'), \\ r(A' \cup A \cup B) &\leq r(A' \cup B). \end{aligned}$$

Therefore, $r(A' \cup B) = r(A' \cup A \cup B)$. Because $r(A' \cup B) \leq |A'| + |B| = r(A \cup B)$, we have that $A' \cup B$ is independent and $B \subseteq N \setminus A'$. $\square$

Lemma 3. At the end of every iteration, marked by the element $j \in [n]$ that was parsed, the set S maintained by Algorithm 7 is maximally independent, that is, $r(S)$ equals the rank of the set $[j]$ that contains all elements parsed so far.

Proof. The algorithm always maintains an independent set and the lemma is true after the first iteration. For the sake of contradiction, let $j+1$ be the first element in submodular order where the lemma is not true. Let S_j denote the set at the end of iteration j (after element j is parsed). By definition of j , $S_j \cup \{j+1\}$ is not independent but $r([j+1]) = r(S_j) + 1$. Using the fact that S_j is rank maximal subset of $r([j])$, there exists an independent set $S'_j \subseteq [j]$ with the same rank as S_j and such that $S'_j \cup \{j+1\}$ is independent. Applying Lemma 2 with $A = S_j$ and $A' = S'_j$, we have $\{j+1\} \in \mathcal{I}_{S_j}$, a contradiction. $\square$

Proof of Theorem 5. Algorithm 7 parses the ground set once. When parsing an element, the algorithm makes at most d queries, resulting in total nd queries.

To show the performance guarantee. Let S_j denote the set S maintained in the algorithm at the beginning of iteration j . Similarly, let R_j denote the set R at the beginning of iteration j . Observe that R_j is the set of all elements that were chosen and later swapped out, prior to iteration j . Therefore, the set $S_j \cup R_j$ grows monotonically and includes all elements selected by the algorithm prior to j . We use S and R to denote the final sets when the algorithm terminates. Let OPT denote the optimal set (and value) and ALG denote the algorithm output (and value). By monotonicity, $f(\text{ALG} \cup \text{OPT}) \geq \text{OPT}$. We will “essentially” show that $f(\text{OPT} | \text{ALG}) \leq 3\text{ALG}$. The main elements that contribute to this upper bound are,

Swap operations: If an element j is taken out of the set at iteration t , then the elements in OPT parsed between j and t , but never included, may have larger marginal value after the swap. We show that the resulting total increase in marginal value is upper bounded by ALG .

Rejection of elements: Any element that is rejected by the algorithm on parsing (due to insufficient marginal value for swapping) may be an element of OPT. Let j denote the index (in submodular order) of such an element. Because j is rejected, we have that $S_j \cup \{j\}$ contains a circuit C_j and the marginal $f(j|S_j \cup R_j)$ is upper bounded by v_i for all $i \in C_j$. We show that the total marginal value $\sum_{j \in \text{OPT} \setminus (S \cup R)} f(j|S_j \cup R_j)$ of elements in OPT rejected by the algorithm is at most 2ALG .

Without loss of generality, we ignore all elements in $N \setminus (\text{OPT} \cup S \cup R)$. Therefore, let $\hat{N} = \text{OPT} \cup S \cup R$ denote our ground set and reindex elements in $\hat{N}$ from one to $|\hat{N}|$ (maintaining submodular order). Similarly, we reindex the sets S_j, R_j so that they continue to denote the set maintained by the algorithm when element $j \in \hat{N}$ is first parsed for every $j \in \{1, 2, \dots, |\hat{N}|\}$. From monotonicity, $\text{OPT} \leq f(\hat{N})$.

Consider an interleaved partition $\{O_\ell, E_\ell\}$ of $\hat{N}$ such that $E(m) = \cup_{\ell \in [m]} E_\ell := S \cup R$ and $O(1) = \cup_{\ell \in [m]} O_\ell := \text{OPT} \setminus (S \cup R)$. Using Corollary 2,

$$\begin{aligned} \text{OPT} &\leq f(E(m)) + \sum_{\ell \in [m]} f(O_\ell | E(\ell - 1)), \\ &\leq f(E(m)) + \sum_{j \in \text{OPT} \setminus (S \cup R)} f(j|S_j \cup R_j), \end{aligned}$$

where the second inequality follows from weak submodular order. Next, we apply Corollary 3(i) to upper bound $f(E(m))$ in terms of ALG. Consider an interleaved partition $\{\bar{O}_\ell, \bar{E}_\ell\}_{\ell \in [\bar{m}]}$ of $E(m)$ such that $\bar{E}(\bar{m}) = S$ and $\bar{O}(1) = R$. We have

$$\begin{aligned} f(E(m)) &\leq \text{ALG} + \sum_{\ell \in [\bar{m}]} f(\bar{O}_\ell | \bar{E}(\ell - 1) \cup \bar{O}(1) \setminus \bar{O}(\ell)), \\ &= \text{ALG} + \sum_{j \in R} f(j|S_j \cup R_j). \end{aligned}$$

We upper bound $\sum_{j \in R} f(j|S_j \cup R_j)$ by ALG and $\sum_{j \in \text{OPT} \setminus (S \cup R)} f(j|S_j \cup R_j)$ by 2ALG . This proves the main claim.

Part I: $\sum_{j \in R} f(j|S_j \cup R_j) \leq \text{ALG}$.

Consider an element $j_1 \in R$ that was added to S_{j_1} without swapping out any element. Because $j_1 \in R$, there exists an element j_2 that later replaced j_1 . Inductively, for $t \geq 2$, let j_t denote the t th element in the chain of swaps $j_1 \rightarrow j_2 \rightarrow \dots \rightarrow j_t$. The chain terminates at an element in S , and we call this a *swap chain*. From the swap criteria in Algorithm 7, we have

$$\sum_{\tau \in [t-1]} f(j_\tau | S_{j_\tau} \cup R_{j_\tau}) \leq f(j_t | R_{j_t} \cup S_{j_t}).$$

Every element in R is part of a unique swap chain, and each chain has a unique terminal element in S .

Therefore,

$$\sum_{j \in R} f(j|S_j \cup R_j) \leq \sum_{i \in S} f(i|S_i \cup R_i).$$

Applying Corollary 3(ii) with $A = S \cup R$ and sets $E_i = \{i\}$ for $i \in S$, we have

$$\sum_{i \in S} f(i|S_i \cup R_i) \leq \text{ALG}. \quad (7)$$

Observe that given a noisy function oracle $\hat{f}$, we have

$$\begin{aligned} \sum_{j \in R} f(j|S_j \cup R_j) &\leq \sum_{j \in R} \hat{f}(j|S_j \cup R_j) + \frac{O(|R|\delta)}{1-\delta} f(\hat{N}), \\ &\leq \sum_{i \in S} \hat{f}(i|S_i \cup R_i) + \frac{O(|R|\delta)}{1-\delta} f(\hat{N}), \\ &\leq \sum_{i \in S} f(i|S_i \cup R_i) + \frac{O(n\delta)}{1-\delta} f(\hat{N}), \end{aligned}$$

where the first and third inequalities follow from the definition of noisy oracle and the second inequality follows from the swap criteria. Thus, $\sum_{j \in R} f(j|S_j \cup R_j) \leq \text{ALG} + O(\frac{n\delta}{1-\delta}) f(\hat{N})$.

Part II: $\sum_{j \in \text{OPT} \setminus (S \cup R)} f(j|S_j \cup R_j) \leq 2 \text{ALG}$.

From Lemma 3, we have that $r(\text{OPT}) \leq r(\text{ALG})$. Suppose there exists an injection ϕ from elements in $\text{OPT} \setminus (S \cup R)$ to elements in S such that

$$f(j|S_j \cup R_j) \leq 2f(\phi(j)|S_{\phi(j)} \cup R_{\phi(j)}), \quad \forall j \in \text{OPT} \setminus (S \cup R).$$

Summing up these inequalities and using (7), we are done. It remains to show that ϕ exists. We do this via a graphical construction inspired by Chekuri et al. (2015).

Consider a graph G with vertices given by $\hat{N}$. To define the edges, recall that every $j \in \text{OPT} \setminus (S \cup R)$ is rejected by the algorithm on parsing. Thus, we have a unique circuit C_j , where $C_j \setminus \{j\} \subseteq S_j$. For our first set of edges, we make a directed edge from j to every element in $C_j \setminus \{j\}$, and we do this for all $j \in \text{OPT} \setminus (S \cup R)$. Next, for every $j \in R$, let C_j represent the chain that causes j to be swapped out in the algorithm. We create a directed edge from j to every element in $C_j \setminus \{j\}$. Graph G has the following properties:

(a) The elements of $\text{OPT} \setminus (S \cup R)$ are *source* vertices with no incoming edges. Elements of S are *sinks* with no outgoing edges.

(b) The neighbors of every node in G form a circuit with the node.

(c) Given arbitrary vertex $j \in \text{OPT} \setminus (S \cup R)$, for every i reachable from j we claim that

$$f(j|S_j \cup R_j) \leq v_i,$$

where v_i corresponds to the value defined in Algorithm 7.

For neighbors of j the claim follows directly from the swap criterion. Also, for any two neighboring vertices $i', i'' \in S \cup R$, we have $v_{i'} \leq v_{i''}$. The general claim follows by using these inequalities for every edge on the path from j to i .

Using (a) and (b), we apply Lemma 4 to obtain an injection ϕ from $\text{OPT} \setminus (S \cup R)$ to S such that for every $j \in \text{OPT} \setminus (S \cup R)$, there exists a path to $\phi(j) \in S$. Then, from (c), we have $f(j|S_j \cup R_j) \leq v_{\phi(j)}$. Recall the notion of a swap chain defined in Part I and let $W(\phi(j))$ denote the set of all preceding elements of the swap chain that terminates at $\phi(j)$. By definition of $v_{\phi(j)}$ and the swap criterion,

$$\begin{aligned} v_{\phi(j)} &= f(\phi(j)|S_{\phi(j)} \cup R_{\phi(j)}) + \sum_{i \in W(\phi(j))} f(i|S_i \cup R_i) \\ &\leq 2f(\phi(j)|S_{\phi(j)} \cup R_{\phi(j)}). \end{aligned}$$

Given a noisy function oracle $\hat{f}$, by the preceding argument there exists an injection ϕ from $\text{OPT} \setminus (S \cup R)$ to S such that $\hat{f}(j|S_j \cup R_j) \leq 2\hat{f}(\phi(j)|S_{\phi(j)} \cup R_{\phi(j)})$, $\forall j \in \text{OPT} \setminus (S \cup R)$. Thus, $\sum_{j \in \text{OPT} \setminus (S \cup R)} \hat{f}(j|S_j \cup R_j) \leq 2\text{ALG} + O(\frac{n\delta}{1-\delta})f(\hat{N})$. Overall, for a noisy oracle, we have $\text{ALG} \geq (1 - O(\frac{n\delta}{1-\delta}))0.25f(\hat{N}) \geq (1 - O(\frac{n\delta}{1-\delta}))0.25\text{OPT}$. $\square$

Lemma 4 (Lemma 30 in Chekuri et al. 2015). *Let $M = (N, \mathcal{I})$ be a matroid, and G a directed acyclic graph over N such that for every nonsink vertex $e \in N$, the outgoing neighbors of e form a circuit with e (alternatively, e is in the span of its neighbors). Let $I \in \mathcal{I}$ be an independent set such that no path in G goes from one element in I to another. Then there exists an injection from I to sink vertices in G such that each $e \in I$ maps into an element reachable from e .*

5. Numerical Experiment

In this section, we discuss results from a small numerical experiment to get a better understanding of the true performance of algorithms for constrained maximization of submodular order functions. We consider the problem of cardinality constrained assortment optimization in the MNL choice model. Recall that the MNL choice model is very widely used in industry, and there are polynomial time algorithms for finding the optimal cardinality constrained assortment in this model. Also, for this problem, the objective function $f(S) = \max_{X \subseteq S} R(S)$ is not submodular but has a submodular order (decreasing order of prices). These features make this setting a good test bench. Our main focus is on comparing the expected revenue of assortments generated by Algorithm 1 (ALG) with the revenue of the optimal assortment (OPT). We also make some observations on the time runtime of ALG.

We consider a setting with $n = 50,000$ products. The price of each product is sampled uniformly randomly from the interval $[15, 25]$. Similarly, we independently sample a parameter μ_i from the interval $[15, 25]$ for

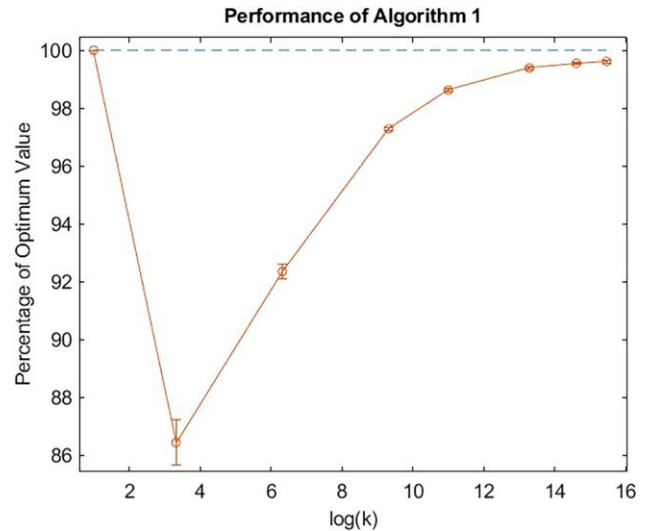
every item $i \in [n]$. This is used to set the attraction parameters in the MNL model:

$$v_i = e^{\mu_i - r_i} \quad \forall i \in [n].$$

This is a standard process for defining an MNL choice model where the parameter μ_i represents the mean utility of item i . We set $v_0 = 1$ for the outside option. We test ALG for different values of cardinality parameter k in the set $\{1, 10, 80, 600, 2,000, 10,000, 25,000, 45,000\}$. For each setting of k , we consider 1,000 random trials where prices and attraction parameters are randomly generated according to the process defined above and we look at the average performance of ALG over these trials. Finally, we show results for ALG when the parameter $\epsilon = 0.1$. Smaller ϵ improves the performance by a few percentage points but the basic insights do not change. We also tried various other values for the other parameters. In particular, we considered different values of n in the range of 50 to 100,000. We also tried sampling prices (and mean utilities) from the wider range of 5 to 50 and also a narrower range of 10 to 15 and observed results that were either similar to what we describe here or were not as informative because ALG was very close to OPT.

We compared ALG with OPT for different values of k . Figure 1 summarizes the comparison with $\log(k)$ on the x axis. Observe that ALG is almost equal to OPT for $k = 1$ and $k \approx n$. This is to be expected as the problem is

Figure 1. (Color online) Plot of $\frac{\text{Average ALG}}{\text{Average OPT}} \times 100$ on the y Axis with $\log(k)$ on the x Axis



Notes. For each value on the x axis, the ratio on the y axis is the average value of ALG over 1,000 instances divided by the average value of OPT over (the same) 1,000 instances. The standard deviation is represented using bars around each data point and becomes negligible for $\log(k) \geq 6$. We did not observe any qualitative change in the plot if the y axis is changed to the average of the ratio $\frac{\text{ALG}}{\text{OPT}}$ as opposed to the ratio of the average values of ALG and OPT.

“easy” for these extremes. In general, ALG is at least 85% of OPT, a significantly higher number than the guarantee of 50% given in Theorem 2. In terms of running time, it took under a minute to execute ALG for all values of k , and it took 2 to 5 minutes to execute the optimal algorithm. We implemented the combinatorial algorithm of Rusmevichientong et al. (2010) to compute the optimal solution. There are various other algorithms for computing OPT (Jagabathula 2014, Avadhanula et al. 2016, Sumida et al. (2021)) that may yield different results for the runtime comparison. Similarly, a more efficient implementation of the algorithms may show different results for runtime comparison. Such a comparison would be interesting but it is beyond the scope of our experiment.

6. Conclusion

We introduced notions of weak and strong submodular order functions and explored the landscape of constrained maximization problems under this structure. We showed that in the value oracle model, no polynomial algorithm can achieve a guarantee better than 0.5 for maximizing function value subject to a cardinality constraint. We proposed algorithms that achieve this best possible guarantee under cardinality and more generally budget constraint. We also gave a 0.25 approximation for the problem under matroid constraint. Applying these results, we obtained improved (and first constant factor) guarantees, with unified algorithms, for several constrained assortment optimization problems. We also observed an intriguing algorithmic connection of submodular order maximization with the problem of streaming submodular optimization. The connection raises some interesting questions about the interplay between structure of set functions and computational considerations such as limited memory.

6.1. Directions for Further Work and Beyond Submodular Order

- What is the best possible approximation guarantee under matroid constraint?
- **Multiple submodular orders:** Consider an order π and its reverse order π_r . Does a function with submodular order along both π and π_r admit better approximations? The upper bound of 0.5 in Theorem 3 applies to functions with multiple submodular orders but not to functions where an order and its reverse are both submodular orders.
- **Unknown submodular order:** Given a function f and the knowledge that it has a submodular order, can we get interesting guarantees for optimization without knowing the order? Are there natural conditions under which the order could be computed using only value queries? What is the computational complexity of verifying that a given order is a submodular order or

certifying the inexistence of (weak) submodular order? Testability problems are hard and not completely understood even for classic submodular functions (Seshadhri and Vondrák 2014).

Recall that knowing the submodular order is not necessary for obtaining algorithmic guarantees. In particular, our algorithm for Markov choice model develops a partial (submodular) order by repeatedly solving an unconstrained version of the problem on smaller and smaller ground sets. Generalizing the scope of such an idea is another interesting direction for future work. Recall that the impossibility result given in Theorem 1 only applies to functions that do not have any (strong or weak) submodular order.

- Are there other applications where objective is not submodular but has submodular order?

Acknowledgments

The author thanks Ali Aouad, Omar El-Housni, Vineet Goyal, and Danny Segev for helpful discussions and comments on early drafts of this work.

Endnote

¹ Other local search techniques exist. For instance, oblivious local search algorithms (Filmus and Ward 2012) may perform swaps to increase the value of a proxy function instead of the objective.

References

- Aouad A, Farias V, Levi R, Segev D (2018) The approximability of assortment optimization under ranking preferences. *Oper. Res.* 66(6):1661–1669.
- Avadhanula V, Bhandari J, Goyal V, Zeevi A (2016) On the tightness of an lp relaxation for rational optimization and its applications. *Oper. Res. Lett.* 44(5):612–617.
- Badanidiyuru A, Vondrák J (2014) Fast algorithms for maximizing submodular functions. *Proc. 25th Annual ACM-SIAM Sympos. on Discrete Algorithms* (SIAM, Philadelphia), 1497–1514.
- Badanidiyuru A, Mirzasoleiman B, Karbasi A, Krause A (2014) Streaming submodular maximization: Massive data summarization on the fly. Macskassy SA, Perlich C, Leskovec J, Wang W, Ghani R, eds. *Proc. 20th ACM SIGKDD Internat. Conf. on Knowledge Discovery and Data Mining* (ACM, New York), 671–680.
- Berbeglia G, Joret G (2020) Assortment optimisation under a general discrete choice model: A tight analysis of revenue-ordered assortments. *Algorithmica* 82(4):681–720.
- Blanchet J, Gallego G, Goyal V (2016) A Markov chain approximation to choice modeling. *Oper. Res.* 64(4):886–905.
- Bradley RA, Terry ME (1952) Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika* 39:324.
- Calinescu G, Chekuri C, Pal M, Vondrák J (2011) Maximizing a monotone submodular function subject to a matroid constraint. *SIAM J. Comput.* 40(6):1740–1766.
- Chakrabarti A, Kale S (2015) Submodular maximization meets streaming: Matchings, matroids, and more. *Math. Programming* 154(1):225–247.
- Chekuri C, Gupta S, Quanrud K (2015) Streaming algorithms for submodular function maximization. *Proc. Internat. Colloquium on Automata, Languages, and Programming* (Springer, Berlin), 318–330.
- Das A, Kempe D (2011) Submodular meets spectral: Greedy algorithms for subset selection, sparse approximation and dictionary selection. Getoor L, Scheffer T, eds. *Proc. 28th Internat. Conf. Machine Learn.* (Omnipress, Madison, WI), 1057–1064.

- Désir A, Goyal V, Zhang J (2021) Technical note—Capacitated assortment optimization: Hardness and approximation. *Oper. Res.* 70(2):893–904.
- Désir A, Goyal V, Segev D, Ye C (2020) Constrained assortment optimization under the Markov chain-based choice model. *Management Sci.* 66(2):698–721.
- Dobzinski S, Nisan N, Schapira M (2010) Approximation algorithms for combinatorial auctions with complement-free bidders. *Math. Oper. Res.* 35(1):1–13.
- El Housni O, Topaloglu H (2023) Joint assortment optimization and customization under a mixture of multinomial logit models: On the value of personalized assortments. *Oper. Res.* 71(4):1197–1215.
- Feige U, Mirrokni VS, Vondrak J (2011) Maximizing non-monotone submodular functions. *SIAM J. Comput.* 40(4):1133–1153.
- Feldman M, Naor J, Schwartz R (2011) A unified continuous greedy algorithm for submodular maximization. *Proc. IEEE 52nd Annual Sympos. on Foundations of Computer Sci.* (IEEE, New York), 570–579.
- Feldman M, Norouzi-Fard A, Svensson O, Zenklusen R (2023) The one-way communication complexity of submodular maximization with applications to streaming and robustness. *J. ACM* 70(4):24:1–24:52.
- Feldman JB, Zhang DJ, Liu X, Zhang N (2022a) Customer choice models vs. machine learning: Finding optimal product displays on Alibaba. *Oper. Res.* 70(1):309–328.
- Feldman M, Liu P, Norouzi-Fard A, Svensson O, Zenklusen R (2022b) Streaming submodular maximization under matroid constraints. Bojańczyk M, Merelli E, Woodruff DP, eds. *Proc. 49th Internat. Colloquium on Automata, Languages, and Programming*, vol. 229 of Leibniz International Proceedings in Informatics (Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany), 59:1–59:20.
- Films Y, Ward J (2012) A tight combinatorial algorithm for submodular maximization subject to a matroid constraint. *Proc. IEEE 53rd Annual Sympos. on Foundations of Computer Sci.* (IEEE, New York), 659–668.
- Huang CC, Kakimura N (2021) Improved streaming algorithms for maximizing monotone submodular functions under a knapsack constraint. *Algorithmica* 83(3):879–902.
- Jagabathula S (2014) Assortment optimization under general choice. Preprint, submitted October 21, <https://dx.doi.org/10.2139/ssrn.2512831>.
- Kök AG, Fisher ML, Vaidyanathan R (2008) Assortment planning: Review of literature and industry practice. *Retail Supply Chain Management*, 99–153.
- Krause A, Singh A, Guestrin C (2008a) Near-optimal sensor placements in gaussian processes: Theory, efficient algorithms and empirical studies. *J. Machine. Learn. Res.* 9(2):235–284.
- Krause A, McMahan HB, Guestrin C, Gupta A (2008b) Robust submodular observation selection. *J. Machine Learn. Res.* 9(12):2761–2801.
- Leskovec J, Krause A, Guestrin C, Faloutsos C, VanBriesen J, Glance N (2007) Cost-effective outbreak detection in networks. Berkhin P, Caruana R, Wu X, eds. *Proc. 13th ACM SIGKDD Internat. Conf. on Knowledge Discovery and Data Mining* (ACM, New York), 420–429.
- Luce RD (1959) *Individual Choice Behavior: A Theoretical Analysis* (Wiley, New York).
- McFadden D (1973) Conditional logit analysis of qualitative choice behavior. Zarembka P, ed. *Frontiers in Econometrics* (Academic Press, New York), 105–142.
- McFadden D, Train K (2000) Mixed MNL models for discrete response. *J. Appl. Econometrics* 15(5):447–470.
- Nemhauser GL, Wolsey LA, Fisher ML (1978) An analysis of approximations for maximizing submodular set functions—I. *Math. Programming* 14(1):265–294.
- Plackett RL (1975) The analysis of permutations. *J. Roy. Statist. Soc. Ser. C Appl. Statist.* 24(2):193–202.
- Rusmevichientong P, Shen ZJM, Shmoys DB (2010) Dynamic assortment optimization with a multinomial logit choice model and capacity constraint. *Oper. Res.* 58(6):1666–1680.
- Seshadhri C, Vondrák J (2014) Is submodularity testable? *Algorithmica* 69(1):1–25.
- Sumida M, Gallego G, Rusmevichientong P, Topaloglu H, Davis J (2021) Revenue-utility tradeoff in assortment optimization under the multinomial logit model with totally unimodular constraints. *Management Sci.* 67(5):2845–2869.
- Sviridenko M (2004) A note on maximizing a submodular set function subject to a knapsack constraint. *Oper. Res. Lett.* 32(1):41–43.
- Talluri K, Van Ryzin G (2004) Revenue management under a general discrete choice model of consumer behavior. *Management Sci.* 50(1):15–33.
- Wei K, Iyer R, Bilmes J (2015) Submodularity in data subset selection and active learning. Bach FR, Blei DM, eds. *Proc. Internat. Conf. on Machine Learn.* (PMLR, New York), 1954–1963.

Copyright 2025, by INFORMS, all rights reserved. Copyright of Management Science is the property of INFORMS: Institute for Operations Research and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.